Programmation Orientée Objet

Partie 0 : introduction

David Parello
david.parello@univ-perp.fr

Université Perpignan Via Domitia

Outline

Objectifs et Organisation

POO ?

Le C++

Premier programme

Bases de la Syntaxe en C++

Objectifs du cours

Ce cours a deux objectifs principaux :

- Introduction à la P.O.O. ainsi qu'à ses spécifités en C++
  - Classes, héritage, amitié, polymorphisme, surcharge d'opérateur, etc. . .
- Initiation au C++ moderne
  - Template, pointeurs intelligents, expressions lambda, itérateurs, etc. . .

Le C++ est un langage qui évolue dans le temps et dont la maîtrise requière de nombreuses années de pratique.

- Ce cours est seulement une **INTRODUCTION**

À vous de continuer à vous perfectionner et de vous tenir au courant des avancées du langage tout au long de votre carrière.

Organisation du cours

Volume horaire : 36h

- 13.5h de CM
- 27h de TP

Évaluation : max{(CC + CT)/2, CT}

- CC : 1 épreuves de 3h (projet : théorie, programmation et oral).
- CT : 1 épreuve de 4h (théorie et programmation).

Référence :

- Programmer en C++ moderne : De C++11 à C++20, Claude Delannoy.

Liens utiles :

- `https://cppreference.com`
- `https://isocpp.org`

Qu'est ce que la POO ?

Programmation Orientée Objet (POO) : paradigme de programmation informatique

- élaboré par Alan Kay dans les années 70
- définition et interactions de briques logicielles objets.

Un objet = un concept, une idée ou une entité du monde physique

- exemple : une voiture, un étudiant, un nombre
- possède une structure interne propre et un comportement défini

Quelques langages objets : Simula, C++, Java, Perl, Python, ...

Idées principales de la POO

Modélisation directe d'objets/concepts du monde réel
un programme contient généralement de très nombreux représentants d'un petit nombre de concepts différents

- exploiter cette redondance
- Réduire l'impact d'une modification/extension d'une partie d'un programme

Encapsulation dans de petites unités qui n'ont que peu de contact avec l'extérieur (objets)

- séparer l'interface et l'implantation de ce que fait l'objet
- l'accès à un objet se fait uniquement via l'interface du haut (fournie à l'utilisateur)
- Définir les interactions des objets entre eux
- développement d'une petite application effort important
- extension d'une application effort faible

Penser le programme en terme d'objets, capables d'interagir entre eux, et pas en lignes de codes qui s'enchaineraient les unes après les autres

La notion d'objet

Un objet = un concept, une idée ou une entité du monde physique

- voiture, personne, étudiant, animal, fenêtre graphique, forme géométrique, . . .

Dans un programme un objet s'apparente à une variable
Un objet est caractérisé par trois notions :

- les attributs : les données de l'objet qui caractérisent son état.
- les méthodes : fonction membre caractérisant le comportement de l'objet, les actions qu'il peut réaliser, les accès/modification des attributs.
- une identité, qui permet de le distinguer de manière unique des autres objets indépendemment de son état.

La notion de classe

Une classe = la structure d'un objet

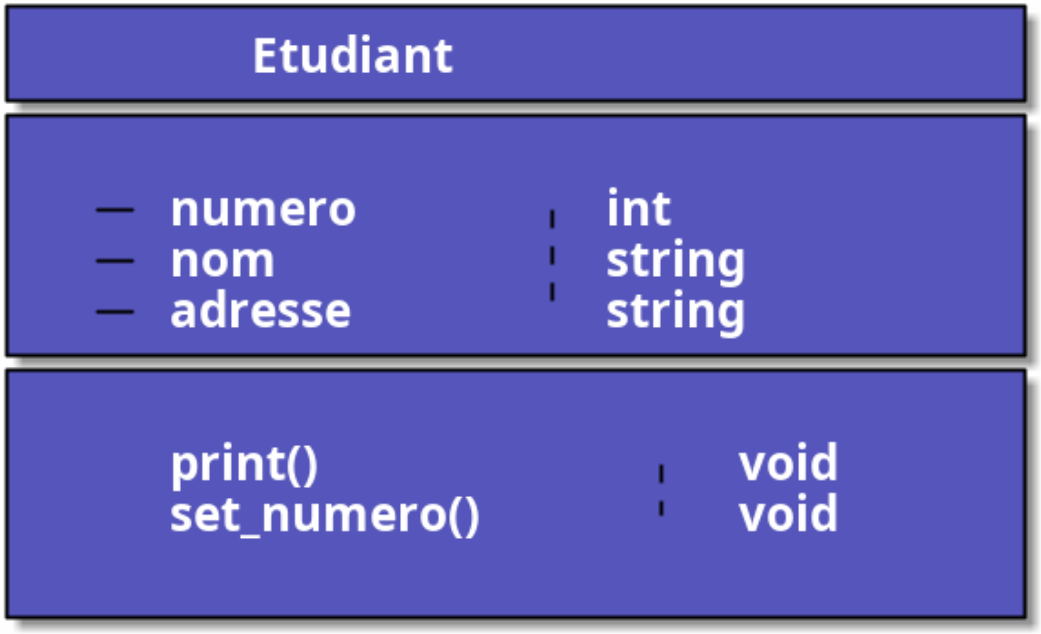
- elle définit son type
- permet de déclarer l'ensemble des entités qui composeront l'objet

Un objet est donc une instanciation d'une classe. Une classe est composée de deux parties

- les attributs : les données membres.
- les méthodes : les fonctions membres

Exemple de classe et d'objet

Modélisation (UML)



Principes fondamentaux de la POO

Encapsulation :

- rassembler données et méthodes au sein d'une structure
- l'implantation de l'objet est cachée à l'utilisateur
- l'accès aux données est protégé
- facilité de maintenance

Héritage :

- création d'une classe à partir d'une classe existante
- La classe fille contient, sous certaines conditions, les attributs et méthodes de la classe mère.
- Évite la duplication du code

Polymorphisme :

- possibilité de redéfinir dans la classe fille les méthodes héritées de la classe mère

C++

Bjarne Stroustrup

C++ makes programming more enjoyable for serious programmers

Historique

- Mis au point par Bjarne Stroustrup (Bell labs) dans les années 80.
- Extension du langage C créé dans les années 70 par Ritchie et Kernighan.

L'ajout de l'approche orientée objet a permis aux programmeurs d'effectuer une transition "douce" de la programmation structurée vers la POO.

- Première norme publiée par l'ANSI et l'ISO en 1998 (C++98)
- **depuis C++03, C++11, C++14, C++17, C++20, C++23, (C++26).**
- enrichissement majeur du langage à partir de C++11 → on parle de **C++ moderne**.
- La Standard Template Library (STL) met à disposition :
  - les chaînes de caractères
  - les conteneurs (vecteurs, listes chaînées, etc...)
  - des algorithmes génériques (recherche, tri, etc...)

Pourquoi utiliser le C++

Le C++ est un langage toujours très populaire (source TIOBE index)

Programming Language	2025	2020	2015	2010	2005	2000	1995	1990	1985
Python	1	3	7	7	7	25	21	-	-
C++	2	4	3	4	3	2	1	2	10
Java	3	1	2	1	2	3	-	-	-
C	4	2	1	2	1	1	2	1	1
C#	5	5	5	6	10	10	-	-	-
JavaScript	6	7	8	9	11	7	-	-	-
Go	7	13	56	183	-	-	-	-	-
Visual Basic	8	12	11	-	-	-	-	-	-
SQL	9	9	-	-	-	-	-	-	-
Delphi/Object Pascal	10	183	12	10	8	-	-	-	-
Fortran	11	33	28	25	16	19	5	3	6
PHP	12	8	6	3	5	24	-	-	-
Ada	18	35	32	26	18	17	6	7	3
Lisp	26	29	27	17	15	9	7	4	2
(Visual) Basic	-	-	-	5	6	4	3	5	4

Plusieurs logiciels de premier plan sont écrits en C++

- Google Chrome, Mozilla Firefox, Microsoft Office, ...

Performance du C++

- moteurs de rendu 3D, jeux vidéos, ...

Premier programme C++17

La syntaxe du C++ est très proche de celle du C :

Hello World

```
1 #include <iostream>
2 int main( )
3 {
4         std::cout << "Goodbye old C++ World!\n";
5         return 0;
6 }
```

Compilation et exécution :

```
1 $>g++ main . c -o main
2 $>./main
3 Goodbye old C++ World!
```

Premier programme C++23

Hello World

rr#+NAME: hworld$_{001cpp23}$

```
#include <print>
int main( )
{
  print("{0} {1} {2} {3}!\n","Hello", "New", "C++23", "World");
  return 0;
}
```

Compilation et exécution :

```
$>g++ -std=c++23 main.c -o main
$>./main
Hello New C++23 World!
```

Compilation Séparée

Rappels sur la compilation séparée

Démonstration

Flux d'entrée et de sortie

L'opérateur `<<` permet d'envoyer de l'information sur un flux de sortie (`std::cout` par exemple)

- `ostream & operator<<(expression)`
- `std::cout` est le flux de sortie standard
- `<<` est surchargé afin de pouvoir prendre des paramètres de type différent (`int, string, double, bool, std::ostream,...`)
- `<<` peut aussi prendre en paramètre des manipulateurs qui permettent de formatter le flux de sortie et d'insérer des caractères spéciaux.
- `std::endl` : insère un retour à la ligne et vide le tampon
- `std::ends` : insère le caractère nul
  - `std::dec/hex/oct` : définit la base pour afficher des valeurs numériques
  - d'autres manipulateurs sont inclus dans le header <iomanip>

Plus d'informations : `https://www.cplusplus.com/reference/ostream/ostream/operator%3C%3C/`

Exemple d'utilisation de `std::cout`

```
#include<iostream>//Contient std::cout, std::endl, std::hex
#include<iomanip>//Contient std::setw(), std::setprecision()
int main()
{
        std::cout << "HelloWorld!\n";
        int x = 42;
        std::cout << "xendecimal:" << x << std::endl;
        std::cout << "xenhexa:" << std::hex << x << std::endl;
        std::cout << std::setw(10);
        floaty = 5.3;
        std::cout << "y=" << y << "\n";
        std::cout << std::setprecision(5);
        std::cout << "y=" << y << std::endl;
        return 0;
}
```

Exécution:

```
HelloWorld!
xendecimal:42
xenhexa:2a
y=    5.3
```

Flux d'entrée et de sortie

L'opérateur **>>** permet de récupérer de l'information sur un flux d'entrée (`std::cin` par exemple)

- `istream &operator>>(expression)`
- `std::cin` est le flux d'entrée standard
- **>>** est surchargé afin de pouvoir prendre des paramètres de type différent (int, string, double, bool, std::istream, . . .)
- Comme **<<**, **>>** peut prendre en paramètre des manipulateurs qui permettent de formatter la lecture du flux d'entrée et d'extraire des caractères spéciaux.

#+ `std::skipws/noskipws` : tient compte (ou pas) des séparateurs en début de l'entrée

- `std::dec/hex/oct` : définit la base pour interpréter les valeurs numériques

Plus d'informations : `https://www.cplusplus.com/reference/istream/istream/operator%3E%3E/`

Exemple d'utilisation de std::cin

```
1  #include<string>
2  #include<iostream>
3  intmain()
4  {
5          intn,a,b;
6          std::cout << "Entrez un nombre en hexa : ";
7          std::cin >> std::hex >> n;
8          std::cout << "Son equivalent decimal est: " << n <<'\n';
9          std::cin >> std::dec;
10         std::cout << "Entrez deux entiers:";
11         std::cin >> a> > b;
12         std::cout << "Vous avez rentre : " << a << "et" << b << '\n
13         std::string text;
14         std::cout << "Tapez quelque chose : ";
15         std::cin >> text;
16         std::cout << "Vous avez tape : " << text << std::endl;
17         return 0;
18 }
```

Lecture flux d'entrée : tampons et séparateurs

- L'ensemble des caractères rentrés par l'utilisateur est validé par la touche entrée.
- Les caractères tapés sont affectés dans une zone mémoire tampon.
- L'opérateur >> explore ces caractères un à un au cours des différentes lectures.
  - Si une partie des caractères n'est pas exploitée par la lecture, ils restent en mémoire jusqu'à la lecture suivante.
  - Si les informations saisies ne suffisent pas pour lire toutes les valeurs voulues, l'opérateur » attendra que l'utilisateur saisisse d'autres caractères.
- Les caractères séparateurs ("white space" en anglais) : `' '`, `'\t'`, `'\n'` ont un rôle particulier
- Par défaut, toute lecture commence par avancer le curseur jusqu'au premier caractère qui ne soit pas un séparateur.
- Ensuite tous les caractères valides sont pris en compte jusqu'au prochain séparateur.

Lecture flux d'entrée : tampons et séparateurs

- Lorsqu'un caractère invalide est rencontré il y a deux cas de figures :
  - On a pu trouver des caractères convenant à l'usage souhaité avant le caractère invalide. Le flux reprendra la lecture à partir du caractère invalide.
  - Aucun caractère valide pour l'usage souhaité n'a été rencontré => la lecture du flux est bloqué et la valeur 0 est assigné à la variable.
- Il est possible de tester l'état d'un flux afin de le débloquer, néanmoins il est préférable d'éviter que cela se produise.

```
int a, b, c;
char d;
std::cin >> a >> b; //25->a=2etb=5
std::cout << a << ' ' << b << "\n"<<std::endl;
std::cin >> a >> b; //258->a=2etb=5
std::cout << a << ' ' << b << "\n" << std::endl;
std::cin >> c >> d; //4r->c=8etd='4'
std::cout << c << ' ' << d << "\n"<<std::endl;
std::cin >> c >> d; //...->c=0etd='4'
std::cout << c << ' ' << d << "\n" << std::endl;
```

Flux d'entrée et de sortie

- les flux de type **`ifstream=/=ofstream`** (disponibles dans le header **`fstream`**) permettent de lire/écrire depuis/sur un fichier.
- Les flux peuvent être ouverts de deux façons différentes :
  - via le constructeur **`std::ifstream`**
    `iflux("chemin/vers/fichier");`
  - via la méthode open() : std::ofstream oflux ;
    `oflux.open("chemin/vers/fichier");`
- Comme pour **`std::cin`**, **`>>`** considère les séparateurs.
  `iflux >> texte` : stockera le mot suivant dans texte.

`iflux.getline(texte,n)` stockera au plus n caractères dans texte (de type char*) et s'arrêtera au premier retour à la ligne rencontré.

- **`getline(iflux,texte)`** stockera la prochaine ligne du fichier dans **`texte`** (de type **`string`**).
- Une fois la lecture/écriture du fichier terminée le flux peut être fermé avec la méthode **`close()`**

Plus d'informations :
`https://www.cplusplus.com/reference/fstream/fstream/`

Gestion des flux avec `std::print` et `std::format`

Recherche documentaire

Implementations

Déclarations

```
// Declarations
int a;
float b;
double c;

// Initializations
int d{1};
float e{2.0};
double f{3.0};
```

Typage

```
int x{7};
std::string txt{"une chaine"};
std::cout << x + txt;

/* Compiler Error */
```

Passage par valeur et par référence

En C les arguments de fonctions sont passés par valeur : la valeur des paramètres est copiée dans des variables locales sur lesquelles la fonction opère.

```
void f(int a){ a = a+1; }
int main(){
        int a=2;
        f(a);
        printf("a=%d\n",a); /*Affichera 2*/
        return 0;
}
```

Le même méchanisme se produit par défaut en C++. En revanche le C++ offre la possibilité de définir des alias sur des variables : les références. Une référence s'utilise comme une variable mais doit être initialisée à la déclaration : `T & x` indique que la variable `x` fait référence à une variable de type `T`.

```
int x = 17;
int &ref = x; // ref et x font reference aumeme int
x = x + 1; //x et ref valent maintenant 18
```

Passage par valeur et par référence

Les références sont principalement utilisées pour la spécification des arguments et des valeurs renvoyées par les fonctions.

```
void f(int& a ) { a = a +1;}
int main(){
   int a = 2;
   f(a);
   std::cout << "a = " << a << std::endl ; /* Affichera 3 */
   return 0;
   }
```

- L'intérêt principal des références est d'éviter la copie les paramètres de fonction dans des variables locales gain d'efficacité.

+À la différence des pointeurs en C qui peuvent être nuls et réassignés, les références doivent toujours pointer sur une variable et ne peuvent pas être réassignés.

```
int x = 16, y = 1;
int& ref = x; // x et ref valent maintenant 16
ref = y; // ref , y ET x valent maintenant 1
y = 2; // ref ET x valent 1 et y vaut 2
```

Variables, références et expressions constantes

- L'inconvénient principal des références est que l'on peut se retrouver à modifier accidentellement une variable.
- En C++, le mot clé const permet à l'utilisateur de définir des entités constantes (fonctions/variables).
- Leur valeur ne pourra pas être modifiée directement
- Elles doivent être initialisées à la déclaration.

```
1
2  const int a = 5;
3  const int v[] = {0,1,2,3};
4  const int b = 2 * a;
5  const int x; // Erreur : x n'est pas initialise
6
7  a = 6; // Erreur: a ne peut pas etre modifiee
8  v[ 2 ] = 5; // Erreur: les v[ i ] ne peuvent pas etre modifies
9
```

Les pointeurs et les constantes

- Le mot clé **`const`** peut être utilisé pour déclarer :
  - un pointeur constant sur un objet : **`T * const x`**
  - un pointeur sur un objet constant : **`T const *x ou const T *x`**
  - un pointeur constant sur un objet constant : **`const T * const x`**

L'adresse d'une variable non constante peut être attribuée à un pointeur de constante. L'adresse d'une constante ne peut pas être attribuée à un pointeur sans restriction. Un pointeur de constante ne peut pas être utilisé pour modifier une variable même si celle ci est non constante.

```
1 int a = 5;
2 const i nt b = 6;
3 const int * pa = &a; // OK
4 *pa = 2; // Erreur : pa est un pointeur sur constante
5 int *pb = &b; // Erreur : b est une valeur constante
```

Variables, références et expressions constantes

- Il est possible de déclarer des références sur des valeurs constantes.
- Celles-ci sont soumis aux mêmes règles que les pointeurs sur constante.

```
int a = 5;
const int& b = a; // OK
a = 6; // OK a est non constant
b = 2; // Erreur b est une reference constante.
```

- L'intérêt principal des références constantes est d'éviter la copie des valeurs dans les paramètres de fonction tout en s'assurant que celle-ci ne seront pas modifiées accidentellement par la fonction.

```
void f( int& a )
{ a = a +1; } // a est modifie
void g( const int& a )
{ a = a +1; } // Erreur : a ne peut pas etre modifie
```

Espaces de noms

- La norme C++ a introduit la notion d'espace de noms qui permet de restreindre la portée des symboles à certaines parties du programme.
- Permet d'éviter les conflits lorsque plusieurs bibliothèques utilisent les mêmes noms de fonctions.
- La définition de l'espace de noms se fait à l'aide du mot clé `namespace`

```
1  namespace 2D
2  {
3          struct Point
4          {
5                  float x, y;
6          }
7  }
8  namespace 3D
9  {
10         struct Point
11         {
12                 float x, y, z;
13         }
14 }
```

Espaces de noms

Pour accéder aux éléments définit à l'intérieur des espaces de noms on utilise

```
2D::Point P; // Point en 2D
3D::Point Q; // Point en 3D
```

- Pour pouvoir accéder aux éléments il faut que la définition de l'espace de noms soit connu (via un **#include**).
- Il est possible de définir un espace de noms en plusieurs fois.

std::string dans **string** **std::cout** dans **iostream**

- Afin d'alléger l'écriture des programmes il est possible d'indiquer au compilateur que les symboles que l'on déclare sont dans un espace de noms à l'aide de l'instruction **using**

```
using namespace 2D;
Point P; // Point en 2D
3D::P oint Q; // Point en 3D
using namespace std;
string a;
cout << "Je peux utiliser cout sans std::" << endl;
```

Fin pour aujourd'hui